

RAPPORT DE PIPELINE

Pipeline 3 — Analyse SAST

SonarQube Static Security Analysis

Projet : SecNotes DevSecOps

Étudiant : Hasna Daoui

Date : 21 May 2026

Pipeline #2437191969 — GitLab CI/CD



Pipeline	#2437191969
Durée	49 min 26 sec
Statut	PASSED ✓
Outil SAST	SonarQube
Branche	main
Commit	bd48c4e0

1. Introduction

Ce rapport présente l'analyse complète du **Pipeline 3** du projet **SecNotes DevSecOps**. Ce pipeline implémente une chaîne CI/CD complète comprenant l'installation des dépendances, l'exécution des tests unitaires, l'analyse statique de sécurité (SAST) via SonarQube, et la construction de l'image Docker de l'application.

L'application *SecNotes* est une application Node.js + MongoDB volontairement vulnérable, conçue dans un cadre pédagogique pour illustrer les bonnes pratiques DevSecOps et la détection automatisée de vulnérabilités.

2. Architecture DevSecOps

L'architecture du pipeline suit le modèle **Shift-Left Security**, intégrant les contrôles de sécurité le plus tôt possible dans le cycle de développement.

Composant	Rôle	Technologie
Source Code	Application Node.js vulnérable (lab)	Node.js 18 + Express
Version Control	Gestion du code source et des branches	GitLab (branche main)
CI/CD Engine	Orchestration des stages du pipeline	GitLab CI/CD Runner local
SAST Engine	Analyse statique du code source	SonarQube (instance locale)
Containerization	Packaging de l'application	Docker
Base de données	Stockage des notes utilisateurs	MongoDB

3. Explication des Stages CI/CD

Le pipeline est composé de 4 stages principaux actifs dans cette exécution :

Stage 1 — install

Ce stage installe toutes les dépendances Node.js du projet via **npm install**. Les modules installés sont mis en cache (node_modules/) pour éviter de les retélécharger lors d'exécutions ultérieures.

Commande exécutée :

```
npm install
```

Artefacts produits : node_modules/ (expire après 1 jour)

Package	Version	Type	Usage
express	4.17.1	prod	Framework HTTP
jsonwebtoken	8.5.1	prod	Authentification JWT
mongoose	^7.6.0	prod	ORM MongoDB
lodash	4.17.19	prod	Utilitaires JS
md5	^2.3.0	prod	Hashage password (vulnérable)

cors	^2.8.5	prod	Gestion CORS
jest	^29.7.0	dev	Framework de tests
supertest	^7.0.0	dev	Tests HTTP
mongodb-memory-server	^10.1.4	dev	DB en mémoire pour tests

Stage 2 — test

Ce stage exécute les tests unitaires et d'intégration avec le framework **Jest**. Les tests couvrent les modules d'authentification (auth.test.js) et la gestion des notes (notes.test.js). Un serveur MongoDB en mémoire (mongodb-memory-server) est utilisé pour isoler les tests de toute base de données externe.

Commande exécutée :

```
npm test (jest --runInBand --detectOpenHandles)
```

Suite de tests	Fichier	Scénarios testés
Auth Tests	tests/auth.test.js	Register, Login, Invalid credentials, JWT token validation
Notes Tests	tests/notes.test.js	CRUD notes, Search, Authorization checks

■■ Note : Ce stage est configuré avec **allow_failure: true**, permettant au pipeline de continuer même en cas d'échec des tests, ce qui est adapté au contexte pédagogique.

Stage 3 — sast (SonarQube) ← Stage Principal

C'est le **stage central de ce pipeline**. L'analyse SAST (Static Application Security Testing) est réalisée via **SonarQube**, un outil open-source de qualité et sécurité du code. Le scanner analyse statiquement l'intégralité du code source dans le dossier `src/` sans exécuter l'application.

Commande exécutée :

```
sonar-scanner \ -Dsonar.host.url=http://10.40.68.161:9000 \ -Dsonar.login=squ_326a3dd7... \ -Dsonar.projectKey=secnotes-vuln-app
```

Configuration SonarQube (sonar-project.properties) :

sonar.projectKey	secnotes-vuln-app
sonar.projectName	SecNotes Vulnerable App
sonar.projectVersion	1.0
sonar.sources	src/
sonar.tests	tests/
sonar.sourceEncoding	UTF-8
sonar.javascript.lcov.reportPaths	coverage/lcov.info

Stage 4 — build (Docker)

Ce stage construit l'image Docker de l'application à partir du **Dockerfile** présent à la racine du projet. L'image est taguée avec le SHA court du commit Git pour assurer la traçabilité.

Commandes exécutées :

```
docker build -t secnotes:$IMAGE_TAG . docker tag secnotes:$IMAGE_TAG secnotes:latest
```

4. Outils Utilisés

Outil	Catégorie	Version	Description
GitLab CI/CD	Orchestration	16.x	Plateforme CI/CD, gestion du pipeline et des runners
SonarQube	SAST	Community Ed.	Analyse statique : bugs, vulnérabilités, code smells, duplications
sonar-scanner	SAST CLI	4.x	Client CLI pour soumettre le code à SonarQube
Node.js	Runtime	18.x	Environnement d'exécution JavaScript côté serveur
npm	Package Mgr	9.x	Gestionnaire de paquets Node.js
Jest	Testing	29.7.0	Framework de tests unitaires et d'intégration
Docker	Container	24.x	Containerisation de l'application
MongoDB	Database	6.x	Base de données NoSQL document-orientée

5. Analyse des Résultats SAST — SonarQube

L'analyse SonarQube a identifié **10 vulnérabilités intentionnelles** dans le code source de l'application SecNotes. Ces vulnérabilités ont été délibérément introduites dans le code pour illustrer les risques de sécurité courants dans les applications web.

[V1] Injection NoSQL (NoSQL Injection)

CRITIQUE

Fichier(s)	src/controllers/authController.js – login() src/controllers/noteController.js – searchNotes()
OWASP Top 10	A03:2021 - Injection
Description	Le login accepte un objet MongoDB comme password au lieu d'une chaîne, permettant à un attaquant de bypasser l'authentification. La recherche de notes parse directement JSON depuis les query parameters sans validation.
Code Vulnérable	<pre>// authController.js - Vulnerable const passwordCriteria = typeof req.body.password === "string" ? md5(req.body.password) : req.body.password; // ← objet passé directement à MongoDB // noteController.js - Vulnerable titleFilter = JSON.parse(rawQuery); // ← injection via query string</pre>
Remédiation	Valider et typer strictement toutes les entrées utilisateur. Utiliser des schémas de validation (Joi, Zod). Ne jamais passer des objets non validés à MongoDB.

[V2] Secret JWT Codé en Dur (Hardcoded Secret)		CRITIQUE
Fichier(s)	src/middleware/auth.js	
OWASP Top 10	A02:2021 - Cryptographic Failures	
Description	La clé secrète JWT est hardcodée dans le code source, exposée dans le dépôt Git. Tout attaquant ayant accès au code peut forger des tokens valides et usurper l'identité de n'importe quel utilisateur, y compris un administrateur.	
Code Vulnérable	<pre>// auth.js - Vulnerable const JWT_SECRET = 'ma-cle-secrete-en-dur-pour-le-lab';</pre>	
Remédiation	Utiliser des variables d'environnement (process.env.JWT_SECRET). Stocker les secrets dans GitLab CI/CD Variables ou un vault (HashiCorp Vault, AWS Secrets Manager).	

[V3] Absence de Validation des Entrées		HAUTE
Fichier(s)	src/controllers/authController.js – register() src/controllers/noteController.js – createNote()	
OWASP Top 10	A03:2021 - Injection / A04:2021 - Insecure Design	
Description	Aucune validation n'est effectuée sur les données reçues lors de l'inscription (username, email, password, role). Un utilisateur peut s'auto-attribuer le rôle 'admin' lors de l'inscription.	
Code Vulnérable	<pre>// authController.js - Vulnerable const user = await User.create({ username: req.body.username, // aucune validation email: req.body.email, password: md5(req.body.password ''), role: req.body.role 'user' // ← privilege escalation });</pre>	
Remédiation	Implémenter une validation côté serveur (express-validator, Joi). Ne jamais laisser le client définir son propre rôle.	

[V4] IDOR — Contrôle d'Accès Manquant		HAUTE
Fichier(s)	src/controllers/noteController.js	
OWASP Top 10	A01:2021 - Broken Access Control	
Description	Les opérations sur les notes (lecture, mise à jour, suppression) ne vérifient pas si la note appartient à l'utilisateur authentifié. Tout utilisateur peut lire, modifier ou supprimer les notes d'un autre utilisateur en connaissant l'ID.	
Code Vulnérable	<pre>// noteController.js - Vulnerable // Aucune vérification note.author === req.user.id const note = await Note.findByIdAndUpdate(req.params.id, req.body, {...});</pre>	
Remédiation	Ajouter un filtre d'autorisation : Note.findOne({_id: req.params.id, author: req.user.id}). Implémenter un middleware RBAC.	

[V6] Hashage Faible MD5 pour les Mots de Passe		HAUTE
Fichier(s)	src/controllers/authController.js	
OWASP Top 10	A02:2021 - Cryptographic Failures	
Description	Les mots de passe sont hashés avec MD5, un algorithme cryptographiquement cassé. MD5 est vulnérable aux attaques par rainbow tables et brute force. Il ne supporte pas de salt ni d'itérations.	
Code Vulnérable	<pre>// authController.js - Vulnerable const md5 = require('md5'); password: md5(req.body.password '')</pre>	
Remédiation	Utiliser bcrypt (avec un coût de 12+), Argon2id, ou scrypt. Ces algorithmes sont conçus spécifiquement pour le hashage de mots de passe (lents, avec salt).	

[V8] Logs Verbeux — Fuite d'Informations Sensibles		MOYENNE
Fichier(s)	src/app.js, src/controllers/*.js, src/middleware/auth.js	
OWASP Top 10	A09:2021 - Security Logging and Monitoring Failures	
Description	L'application journalise le corps complet des requêtes HTTP (incluant mots de passe), les tokens JWT générés, et les stack traces d'erreurs dans les réponses API. Ces informations exposent des données sensibles dans les logs et aux clients.	
Code Vulnérable	<pre>// app.js - Log du body complet (inclut les mots de passe) console.log('[request] body:', req.body); // auth.js - Token exposé dans les logs console.log(`[login] success for \${user.username} with token: \${token}`); // Stack trace retournée au client res.status(500).json({ error: err.message, stack: err.stack });</pre>	
Remédiation	Utiliser un logger structuré (Winston, Pino) avec filtrage des champs sensibles. Ne jamais retourner de stack traces au client en production.	

[V10] CORS Trop Permissif		MOYENNE
Fichier(s)	src/app.js	
OWASP Top 10	A05:2021 - Security Misconfiguration	
Description	La configuration CORS autorise toutes les origines (*), ce qui permet à n'importe quel site web malveillant de faire des requêtes cross-origin vers l'API et d'exploiter les cookies de session des utilisateurs.	
Code Vulnérable	<pre>// app.js - Vulnerable app.use(cors({ origin: '*' }));</pre>	
Remédiation	Restreindre les origines autorisées à la liste blanche des domaines légitimes : <pre>app.use(cors({ origin: 'https://secnotes.example.com' }));</pre>	

6. Résumé des Vulnérabilités Détectées

ID	Vulnérabilité	Sévérité	OWASP	Fichier Principal
V1	Injection NoSQL	CRITIQUE	A03:2021	authController.js / noteController.js
V2	Secret JWT Hardcodé	CRITIQUE	A02:2021	middleware/auth.js
V3	Absence Validation Entrées	HAUTE	A03 / A04	authController.js
V4	IDOR - Contrôle d'Accès	HAUTE	A01:2021	noteController.js
V6	Hashage MD5 Faible	HAUTE	A02:2021	authController.js
V8	Logs Verbeux / Fuite Info	MOYENNE	A09:2021	app.js / controllers
V10	CORS Trop Permissif	MOYENNE	A05:2021	app.js

Distribution par sévérité :

Sévérité	Nombre	Pourcentage
CRITIQUE	2	28.6%
HAUTE	3	42.9%
MOYENNE	2	28.6%
TOTAL	7	100%

7. Conclusion

Le **Pipeline 3** du projet SecNotes a été exécuté avec succès, validant l'ensemble des 4 stages : installation, tests, analyse SAST et build Docker. Le pipeline a fonctionné en **49 minutes et 26 secondes** et a obtenu le statut **PASSED** (#2437191969).

L'analyse SAST via **SonarQube** a mis en évidence **7 vulnérabilités de sécurité** réparties entre 2 critiques, 3 hautes et 2 moyennes, couvrant les catégories OWASP Top 10 les plus fréquentes : injection NoSQL, secrets hardcodés, IDOR, hashage faible, logs verbeux et CORS permissif.

Ces vulnérabilités ont été **intentionnellement introduites** dans l'application SecNotes dans un cadre pédagogique. Ce pipeline démontre l'efficacité de l'approche **Shift-Left Security** : en intégrant l'analyse de sécurité dès la phase CI/CD, les problèmes sont détectés avant le déploiement en production.

Points clés :

- SonarQube identifie efficacement les vulnérabilités de type injection, secrets exposés et mauvaises pratiques cryptographiques.
- L'intégration SAST dans le pipeline CI/CD permet une détection précoce et automatique.
- Les résultats doivent être complétés par une analyse DAST (OWASP ZAP — stage zap) pour une couverture complète.
- La configuration `allow_failure: true` sur les stages sensibles assure la continuité du pipeline en contexte de lab.

